

**LAPORAN PRAKTIKUM BSD 2 HARI 5
(AGGREGATION FRAMEWORK)**



**MUHAMMAD ZAIDAN HABIBI
(225443020)**



**1AEC-1
TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
POLITEKNIK MANUFAKTUR BANDUNG
TAHUN AJARAN 2025/2026**

PENDAHULUAN

Praktikum ini bertujuan untuk membantu saya untuk memahami dan mengimplementasikan Aggregation Framework pada MongoDB dalam menganalisis data produksi dan kualitas. Melalui rangkaian stage yang tersusun dalam sebuah pipeline, proses pengolahan data dapat dilakukan secara sistematis, mulai dari penyaringan, pengelompokan, hingga perhitungan berbagai metrik kinerja. Pendekatan ini memungkinkan data - data mentah diolah langsung di dalam database menjadi informasi yang lebih terstruktur, ringkas, dan mudah dianalisis. Selain itu, pipeline yang dirancang juga mendukung penggabungan data antar sumber, pembuatan beberapa laporan secara bersamaan, serta simulasi perhitungan untuk evaluasi kualitas. Dengan demikian, dari praktikum ini diharapkan saya mampu melakukan analisis kinerja mesin, mengamati tren, serta mengidentifikasi permasalahan secara lebih efisien tanpa memerlukan pemrosesan tambahan di sisi aplikasi.

Diagram Koleksi

Data Dummy	
_id	ObjectId
mesin	String
tanggal	Date
shift	Number
Produksi	
target	Number
actual_ok	Number
actual_reject	Number
Durasi (Menit)	
durasi_tersedia_menit	Number
Durasi_operasi_menit	Number

A. Membuat Validasi

```
use studi_kasus_oe
db.createCollection("produksi_harian", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["mesin", "tanggal", "shift", "target", "actual_ok", "actual_reject",
"durasi_operasi_menit", "durasi_tersedia_menit"],
      properties: {
        mesin: {
          bsonType: "string",
          description: "Kode mesin (M01 - M10)"
        },
        tanggal: {
          bsonType: "date",
          description: "Tanggal produksi"
        },
        shift: {
          bsonType: "int",
          description: "Shift (1,2,3)"
        },
        target: {
          bsonType: "int",
          description: "Target produksi (unit)"
        },
        actual_ok: {
          bsonType: "int",
          description: "Hasil baik aktual"
        },
        actual_reject: {
          bsonType: "int",
          description: "Hasil cacat"
        },
        durasi_operasi_menit: {
          bsonType: "int",
          description: "Waktu operasi aktual (menit)"
        },
        durasi_tersedia_menit: {
          bsonType: "int",
          description: "Waktu tersedia (menit)"
        }
      }
    }
  }
})
```

B. Koleksi Data Dummy

```
db.produksi_harian.drop()

var docs = [];

var mesinList = []; for (let i=1;i<=10;i++)
mesinList.push("M"+i.toString().padStart(2,'0'));

var start = new Date(2026,3,1); // April
for (let i=0; i<1000; i++) {

    var tgl = new Date(start.getTime() + Math.floor(Math.random()*90)*86400000);

    var shift = Math.floor(Math.random()*3)+1;

    var target = Math.floor(Math.random()*301)+200;

    var actual_ok = Math.floor(target * (0.6 + Math.random()*0.4));

    var actual_reject = Math.floor(actual_ok * Math.random()*0.15);

    var durasi_tersedia = 480;

    var durasi_operasi = Math.floor(durasi_tersedia * (0.7 + Math.random()*0.3));

    docs.push({

        mesin: mesinList[Math.floor(Math.random()*10)],

        tanggal: tgl,

        shift: shift,

        target: target,

        actual_ok: actual_ok,

        actual_reject: actual_reject,

        durasi_operasi_menit: durasi_operasi,

        durasi_tersedia_menit: durasi_tersedia

    });

    if (docs.length === 500) { db.produksi_harian.insertMany(docs); docs = []; }

}

if (docs.length) db.produksi_harian.insertMany(docs);
```

Verifikasi Data

```
db.produksi_harian.countDocuments()

db.produksi_harian.findOne()
```

C. Menghitung OEE per Mesin

C.1. Rumus dan Stage

Saddfields - menambahkan field perhitungan parsial:

- $\text{availability} = \text{durasi_operasi_menit} / \text{durasi_tersedia_menit}$
`availability: { $divide: ["$total_op", "$total_avail"] }`
- $\text{performance} = \text{actual_ok} / \text{target}$ (asumsi sederhana)
`performance: { $divide: ["$total_ok", "$total_target"] },`
- $\text{quality} = \text{total_ok} / (\text{total_ok} + \text{total_reject})$
`quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }`

C.2. Pipeline Lengkap

```
db.produksi_harian.aggregate([  
  // Tahap 1: Tidak perlu $match karena ingin semua data  
  // Tahap 2: Group by mesin dan bulan  
  { $group: {  
    _id: {  
      mesin: "$mesin",  
      bulan: { $month: "$tanggal" },  
      tahun: { $year: "$tanggal" }  
    },  
    total_target: { $sum: "$target" },  
    total_ok: { $sum: "$actual_ok" },  
    total_reject: { $sum: "$actual_reject" },  
    total_op: { $sum: "$durasi_operasi_menit" },  
    total_avail: { $sum: "$durasi_tersedia_menit" }  
  }},  
  // Tahap 3: Hitung OEE  
  { $project: {  
    _id: 0,  
    mesin: "$_id.mesin",  
    bulan: "$_id.bulan",  
    total_target: "$total_target",  
    total_ok: "$total_ok",  
    total_reject: "$total_reject",  
    total_op: "$total_op",  
    total_avail: "$total_avail",  
    performance: { $divide: ["$total_ok", "$total_target"] },  
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },  
    availability: { $divide: ["$total_op", "$total_avail"] }  
  }}])
```

```

    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 4: Urutkan berdasarkan OEE terendah
  { $sort: { OEE: 1 } }
])

```

Output:

```

total_target: 450,
total_ok: 332,
total_reject: 31,
mesin: 'M07',
bulan: 3,
tahun: 2026,
availability: 0.7041666666666667,
performance: 0.7377777777777778,
quality: 0.9146005509641874,
OEE: 0.4751519232731354

```

Penjelasan:

Di dalam **\$group** terdapat data mesin, tanggal, jumlah target, ok, reject, serta durasi operasi dan tersedia. Selanjutnya, **\$project** untuk menghitung nilai, performance, quality, dan OEE. Terakhir **\$sort** berguna untuk mengurutkan data yang ingin ditampilkan, 1 untuk ascending dan -1 untuk descending.

D. Menemukan Mesin OEE di Bawah 80%

```
db.produksi_harian.aggregate([

  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},

  // Tahap 3: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },

```

```

        { $divide: [ "$total_ok", { $add: [ "$total_ok", "$total_reject" ] } ] }
    ]
}
}},
// Tahap 4: Filter OEE kurang dari 80%
{ $match: { OEE: { $lt: 0.80 } } },
// Tahap 5: Urutkan berdasarkan OEE terendah
{ $sort: { OEE: 1 } }
])

```

Output:

```

total_target: 450,
total_ok: 332,
total_reject: 31,
mesin: 'M07',
bulan: 3,
tahun: 2026,
availability: 0.7041666666666667,
performance: 0.7377777777777778,
quality: 0.9146005509641874,
OEE: 0.4751519232731354

```

Penjelasan:

Untuk **\$group**, **\$project**, **\$sort** semuanya masih sama, tetapi yang membedakan dari sintaks sebelumnya yaitu tambahan **\$match** untuk memfilter OEE dibawah 80%.

E. Distribusi Jumlah Produksi per Shift dalam Bucket

```

db.produksi_harian.aggregate([
  { $bucket: {
    groupBy: "$actual_ok",
    boundaries: [0, 200, 300, 400, 600], // 0-199, 200-299, 300-399, 400-600
    default: "di atas 600",
    output: {

```

```

        count: { $sum: 1 },
        rata_reject: { $avg: "$actual_reject" }
    }
  }}
})

```

Output:

```

_id: 0,
count: 169,
rata_reject: 11.881656804733728

```

E.1. Dipisah per Shift

```

db.produksi_harian.aggregate([
  { $group: {
    _id: "$shift",
    data: { $push: "$actual_ok" } // kumpulkan nilai
  }},
  // Kemudian di aplikasi atau gunakan operator $bucket di sub-pipeline (tidak bisa
  langsung karena $bucket butuh array dokumen)
])

```

Output:

```

_id: 3,
data: [
  207,
  451,
  461,
  271,
  327,
  294,
  298,
  264,

```

Penjelasan:

\$group hanya untuk mengumpulkan data shift, kekurangannya dalam program ini terdapat pada belum dijadikan **\$bucket**, sehingga data yang ditampilkan tidak rapih.

E.2. Alternatif Menggunakan \$facet untuk Menjalankan Pipeline Bucket

```
db.produksi_harian.aggregate([
  { $facet: {
    "shift1": [
      { $match: { shift: 1 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default:
"600+", output: { count: {$sum:1} } } }
    ],
    "shift2": [
      { $match: { shift: 2 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default:
"600+", output: { count: {$sum:1} } } }
    ],
    "shift3": [
      { $match: { shift: 3 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default:
"600+", output: { count: {$sum:1} } } }
    ]
  }}
])
```

Output:

```
shift3: [
  {
    _id: 0,
    count: 52
  },
  {
    _id: 200,
    count: 156
  }
]
```

E.3 Lebih Sederhana

```
db.produksi_harian.aggregate([
  { $group: { _id: "$shift", total_produksi: { $sum: "$actual_ok" } } } ])
```

Output:

```
_id: 1,  
total_produksi: 95479
```

F. Optimisasi dan Analisis

- **Indeks:** Buat indeks compound untuk tanggal dan mesin agar query OEE lebih cepat.

```
db.produksi_harian.createIndex({ mesin: 1, tanggal: 1 })
```

- **explain():** Jalankan pipeline OEE dengan `.explain("executionStats")` dan perhatikan apakah indeks terpakai (walaupun \$group tanpa \$match mungkin tetap melakukan COLLSCAN, namun dengan indeks bisa mempercepat pengelompokan jika ada filter tanggal).
- **Penyempurnaan OEE:** Rumus di atas menggunakan rasio `actual_ok / target` untuk performance. Di dunia nyata, performance dihitung dari $(\text{Total Parts} / (\text{Run Time} \times \text{Ideal Speed}))$. Dengan data yang ada, kita bisa menyesuaikan asumsi.

G. Latihan Tambahan

1. Tren Bulanan: Modifikasi pipeline agar menampilkan OEE per bulan untuk semua mesin, lalu urutkan bulan.

```
db.produksi_harian.aggregate([  
  // Tahap 1: Group by mesin dan bulan  
  { $group: {  
    _id: {  
      mesin: "$mesin",  
      bulan: { $month: "$tanggal" },  
      tahun: { $year: "$tanggal" }  
    },  
    total_target: { $sum: "$target" },  
    total_ok: { $sum: "$actual_ok" },  
    total_reject: { $sum: "$actual_reject" },  
    total_op: { $sum: "$durasi_operasi_menit" },  
  },  
])
```

```

    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 2: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 3: Urutkan berdasarkan bulan (dari bulan tertua ke terbaru)
  { $sort: { tahun: 1, bulan: 1, mesin: 1 } } ] )

```

Output:

```

total_target: 475,
total_ok: 440,
total_reject: 58,
mesin: 'M02',
bulan: 3,
tahun: 2026,
availability: 0.8083333333333333,
performance: 0.9263157894736842,
quality: 0.8835341365461847,
OEE: 0.6615655604875643

```

Penjelasan:

Kunci terpenting agar dapat mengurutkan bulan dan OEE, diperlukan sintaks **\$sort** diakhir untuk mengatur bulan, tahun, mesin, dan OEE dengan menggunakan 1 (ascending atau dari bawah ke atas).

2. Mesin Terburuk: Tampilkan 3 mesin dengan OEE terendah di bulan Juni 2026 saja (gunakan \$match di awal untuk filter bulan).

```
db.produksi_harian.aggregate([
  // Tahap 1: Match Bulan Juni atau 6
  { $match: {
    $expr: { $eq: [{ $month: "$tanggal" }, 6] }
  }},
  // Tahap 2: Group by mesin dan bulan
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 3: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
```

```

    total_reject: 1,
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 4: Urutkan OEE
  { $sort: { OEE: 1 } },
  // Tahap 5: Ambil hanya 3 data
  { $limit: 3 }
])

```

Output:

```

total_target: 7308,
total_ok: 5548,
total_reject: 400,
mesin: 'M05',
bulan: 6,
tahun: 2026,
OEE: 0.5870777734976509

```

Penjelasan:

Karena anda harus hanya menampilkan bulan itu saja, diperlukan **\$match** untuk memfilter bulan yang harus ditampilkan, seperti pada kasus ini anda diiminta hanya menampilkan bulan juni atau 6. Terakhir anda harus menampilkan 3 data terburuk, maka gunakan **\$limit** untuk mengatur total data yang ditampilkan.

3. Kinerja Harian: Hitung OEE per mesin per hari (guna `$dateToString` untuk ekstrak tanggal).

```

db.produksi_harian.aggregate([
  // Tahap 1: Group by mesin dan tanggal (tanpa jam)

```

```

{ $group: {
  _id: {
    mesin: "$mesin",
    tanggal: { $dateToString: { format: "%Y-%m-%d", date: "$tanggal" } }
  },
  total_target: { $sum: "$target" },
  total_ok: { $sum: "$actual_ok" },
  total_reject: { $sum: "$actual_reject" },
  total_op: { $sum: "$durasi_operasi_menit" },
  total_avail: { $sum: "$durasi_tersedia_menit" }
}},
// Tahap 2: Hitung OEE
{ $project: {
  _id: 0,
  mesin: "$_id.mesin",
  tanggal: "$_id.tanggal",
  total_ok: 1,
  total_target: 1,
  total_reject: 1,
  availability: { $divide: ["$total_op", "$total_avail" ] },
  performance: { $divide: ["$total_ok", "$total_target" ] },
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject" ] } ] },
  OEE: {
    $multiply: [
      { $divide: ["$total_ok", "$total_target" ] },
      { $divide: ["$total_op", "$total_avail" ] },
      { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject" ] } ] }
    ]
  }
}},
// Tahap 3: Urutkan dari OEE terendah (opsional)
{ $sort: { tanggal: 1, mesin: 1 } }
})

```

Output:

```
total_target: 475,  
total_ok: 440,  
total_reject: 58,  
mesin: 'M02',  
tanggal: '2026-03-31',  
availability: 0.8083333333333333,  
performance: 0.9263157894736842,  
quality: 0.8835341365461847,  
OEE: 0.6615655604875643  
}  
{  
total_target: 958,  
total_ok: 726,  
total_reject: 34,  
mesin: 'M05',  
tanggal: '2026-03-31',  
availability: 0.8430555555555556,  
performance: 0.7578288100208769,  
quality: 0.9552631578947368,  
OEE: 0.6103097873860016  
}  
{  
total_target: 730,  
total_ok: 507,  
total_reject: 28,  
mesin: 'M06',  
tanggal: '2026-03-31',  
availability: 0.8041666666666667,  
performance: 0.6945205479452055,  
quality: 0.9476635514018692,  
OEE: 0.5292798297273077
```

Penjelasan:

\$group untuk mengelompokkan data mesin dan tanggal lalu menjumlahkan semua metric. **\$project** untuk menghitung availability, performance, quality, dan OEE per mesin per hari. Terakhir **\$sort** untuk mengurutkan tanggal ascending dan nama mesin.

4. Simulasi Perbaikan: Jika quality harus minimal 95%, tampilkan mesin yang tidak memenuhi kriteria tersebut.

```
db.produksi_harian.aggregate([
  // Tahap 1: Group by mesin dan bulan
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 2: Hitung OEE dan quality
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_reject: 1,
    quality: {
      $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ]
    },
  }},
])
```

```

    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target" ] },
        { $divide: ["$total_op", "$total_avail" ] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject" ] } ] }
      ]
    }
  }},
  // Tahap 3: Filter quality kurang dari 95%
  { $match: { quality: { $lt: 0.95 } } },
  // Tahap 4: Urutkan dari quality terendah
  { $sort: { quality: 1 } }
})

```

Output:

```

    total_ok: 440,
    total_reject: 58,
    mesin: 'M02',
    bulan: 3,
    tahun: 2026,
    quality: 0.8835341365461847,
    OEE: 0.6615655604875643
  }
  {
    total_ok: 563,
    total_reject: 63,
    mesin: 'M09',
    bulan: 3,
    tahun: 2026,
    quality: 0.8993610223642172,
    OEE: 0.5724130796410498
  }
  {
    total_ok: 332,
    total_reject: 31,
    mesin: 'M07',
    bulan: 3,
    tahun: 2026,
    quality: 0.9146005509641874,
    OEE: 0.4751519232731354
  }
}

```

Penjelasan:

\$group untuk mengelompokkan data mesin dan tanggal lalu menjumlahkan semua metric. **\$project** untuk menghitung availability, performance, quality, dan OEE per mesin per hari. **\$match** memfilter hanya mesin dengan kualitas di bawah 95%. Terakhir **\$sort** untuk mengurutkan quality secara ascending.

Tugas Mandiri Pertemuan 5

Tahap 1 - Script Python

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

NIM = "225443020"
client = MongoClient('mongodb://localhost:27017')
db = client[f'tugas5_{NIM}']
db.inspeksi.drop()
db.target_kualitas.drop()

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
batch_ids = [f"B{str(i).zfill(4)}" for i in range(1, 2001)]
inspektor_list = ["Andi", "Budi", "Citra", "Dian", "Eka"]
jenis_cacat_options = ["gores", "retak", "bengkok", "warna", "dimensi", "pori"]

# Generate data inspeksi (1500 dokumen)
docs = []
start_date = datetime(2026, 4, 1, 6, 0)
for i in range(1500):
    batch = random.choice(batch_ids)
    mesin = random.choice(mesin_list)
    tanggal = start_date + timedelta(minutes=random.randint(0, 60*24*90)) # 3 bulan
    shift = 1 if tanggal.hour < 14 else (2 if tanggal.hour < 22 else 3)
    jumlah = random.randint(100, 500)
```

```

reject = int(jumlah * random.uniform(0, 0.10)) # 0-10% cacat

jenis = random.sample(jenis_cacat_options, k=random.randint(1, 3))

doc = {
    "batch_id": batch,
    "mesin": mesin,
    "tanggal": tanggal,
    "shift": shift,
    "inspektor": random.choice(inspektor_list),
    "jumlah_diperiksa": jumlah,
    "cacat_ditemukan": reject,
    "jenis_cacat": jenis
}

docs.append(doc)

if len(docs) >= 500:
    db.inspeksi.insert_many(docs)
    docs.clear()

if docs:
    db.inspeksi.insert_many(docs)

print("Data inspeksi selesai. Total:", db.inspeksi.count_documents({}))

# Generate target kualitas (setiap batch punya target cacat maksimal 5% dari rata-rata ukuran batch)
targets = []
for i in range(1, 2001):
    batch = f"B{str(i).zfill(4)}"

    # target cacat acak 1-15
    target_cacat = random.randint(1, 15)

    targets.append({"batch_id": batch, "target_maks_cacat": target_cacat})

    if len(targets) >= 500:
        db.target_kualitas.insert_many(targets)
        targets.clear()

if targets:

```

```
db.target_kualitas.insert_many(targets)
print("Data target kualitas selesai. Total:", db.target_kualitas.count_documents({}))
```

Penjelasan:

Anda harus menginput data dari vscode dan menggunakan python. Jangan lupa anda pastikan pymongo sudah terinstall, jika belum anda ketikan pip install pymongo di terminal.

Tahap 2 - Memastikan Data Siap

```
db.inspeksi.countDocuments() // harus >= 1000
db.target_kualitas.countDocuments() // harus >= 50
db.inspeksi.findOne()
db.target_kualitas.findOne()
```

Output:

```
_id: ObjectId('69f84dadfb8212983e28cd14'),
batch_id: 'B0001',
target_maks_cacat: 6
```

Tahap 3 - Percepat Agregasi

```
db.inspeksi.createIndex({ batch_id: 1 })
db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })
db.target_kualitas.createIndex({ batch_id: 1 })
```

```

_id: ObjectId('69f84dadfb8212983e28cad8'),
batch_id: 'B0850',
mesin: 'M01',
tanggal: 2026-05-09T17:47:00.000Z,
shift: 2,
inspektor: 'Dian',
jumlah_diperiksa: 482,
cacat_ditemukan: 48,
jenis_cacat: [
  'bengkok'
],
persen_cacat: 9.95850622406639

_id: ObjectId('69f84dadfb8212983e28cb8a'),
batch_id: 'B0221',
mesin: 'M06',
tanggal: 2026-05-02T01:04:00.000Z,
shift: 1,
inspektor: 'Budi',
jumlah_diperiksa: 322,
cacat_ditemukan: 32,
jenis_cacat: [
  'retak',
  'warna',
  'bengkok'
]

```

Penjelasan:

Terdapat kesalahan karena `.limit(10)` ditambahkan di luar pipeline `aggregate()`, padahal pada operasi agregasi MongoDB pembatasan data harus ditulis sebagai stage `$limit` di dalam pipeline, bukan sebagai method chaining seperti pada `find()`.

Tahap 4 – Persentase Cacat per Batch

```

db.inspeksi.aggregate([
  {
    $addFields: {
      persen_cacat: {
        $multiply: [
          { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
          100
        ]
      }
    }
  }
],

```

```
{ $sort: { persen_cacat: -1 } },
  { $limit: 10 }
])
```

Output:

```
_id: ObjectId('69f84dadfb8212983e28cd11'),
batch_id: 'B1543',
mesin: 'M09',
tanggal: 2026-05-19T01:22:00.000Z,
shift: 1,
inspektor: 'Citra',
jumlah_diperiksa: 326,
cacat_ditemukan: 32,
jenis_cacat: [
  'pori',
  'dimensi',
  'warna'
],
```

Penjelasan:

Kesalahan pada query tersebut adalah penggunaan `.limit(10)` di luar pipeline `aggregate()`, karena pada operasi agregasi MongoDB pembatasan jumlah data harus ditulis sebagai stage `{ $limit: 10 }` di dalam array pipeline, bukan sebagai method chaining di akhir.

Tahap 5 – Gabung dengan Target dan Tentukan Status Batch

```
db.inspeksi.aggregate([
  {
    $lookup: {
      from: "target_kualitas",
      localField: "batch_id",
      foreignField: "batch_id",
      as: "target_info"
    }
  },
  { $unwind: "$target_info" },
  {
```

```

    $addFields: {
      persen_cacat: {
        $multiply: [
          { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
          100
        ]
      },
      target_maks: "$target_info.target_maks_cacat"
    }
  },
  {
    $addFields: {
      status: {
        $cond: {
          if: { $gt: ["$persen_cacat", "$target_maks"] },
          then: "NOT OK",
          else: "OK"
        }
      }
    }
  },
  {
    $project: {
      _id: 0,
      batch_id: 1,
      mesin: 1,
      persen_cacat: 1,
      target_maks: 1,
      status: 1,
      selisih: { $subtract: ["$persen_cacat", "$target_maks"] }
    }
  },
  { $sort: { selisih: -1 } },

```

```
{ $limit: 10 } // dipindahkan ke dalam pipeline
])
```

Output:

```
{
  batch_id: 'B1543',
  mesin: 'M09',
  persen_cacat: 9.815950920245399,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.815950920245399
}
{
  batch_id: 'B1308',
  mesin: 'M04',
  persen_cacat: 9.554140127388536,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.554140127388536
}
{
  batch_id: 'B0070',
  mesin: 'M09',
  persen_cacat: 9.411764705882353,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.411764705882353
}
```

Penjelasan:

Sama seperti sebelumnya, kesalahan pada query tersebut adalah penggunaan `.limit(10)` di luar pipeline `aggregate()`, karena dalam MongoDB pembatasan jumlah hasil agregasi harus ditulis sebagai stage `{ $limit: 10 }` di dalam pipeline, bukan sebagai method chaining di akhir.

Tahap 6 – Jumlah Batch NOT OK per Mesin per Minggu

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
```

```

    persen_cacat: { $multiply: [ { $divide: [ "$cacat_ditemukan", "$jumlah_diperiksa" ] },
100 ] }

    }},

    { $addField: {

        status: { $cond: { if: { $gt: [ "$persen_cacat", "$target_info.target_maks_cacat" ] },
then: "NOT OK", else: "OK" } }

    }},

    { $match: { status: "NOT OK" } },

    { $group: {

        _id: {

            mesin: "$mesin",

            minggu: { $week: "$tanggal" },

            tahun: { $year: "$tanggal" }

        },

        jumlah_NOT_OK: { $sum: 1 }

    }},

    { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }

}]

```

Output:

```

{
  _id: {
    mesin: 'M01',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 2
}
{
  _id: {
    mesin: 'M03',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M04',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 3
}

```

Penjelasan:

Query tersebut berfungsi untuk menggabungkan data inspeksi dengan target kualitas menggunakan **\$lookup**, kemudian menghitung persentase cacat (persen_cacat) dan menentukan status kualitas (OK atau NOT OK) berdasarkan perbandingan dengan target. Setelah itu, data difilter hanya untuk yang berstatus NOT OK, lalu dikelompokkan berdasarkan mesin, minggu, dan tahun untuk menghitung jumlah kasus NOT OK tiap kelompok, dan akhirnya diurutkan berdasarkan tahun, minggu, dan mesin.

Tahap 8 – Ringkasan Bulanan dengan \$facet

```
db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa" ] },
    100 ] },
    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan",
"$jumlah_diperiksa" ] }, 100 ] }, "$target_info.target_maks_cacat" ] }, then: "NOT OK", else:
"OK" } }
  }},
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
```

```

    { $match: { status: "NOT OK" } },
    { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
    { $sort: { jumlah: -1 } },
    { $limit: 3 }
  ]
}}
})

```

Output:

```

total_batch: [
  {
    count: 469
  }
],
rata_cacat: [
  {
    _id: null,
    avg_cacat: 4.694921513283955
  }
],
mesin_terburuk: [
  {
    _id: 'M07',
    jumlah: 16
  },
  {
    _id: 'M06',
    jumlah: 15
  },
  {
    _id: 'M02',
    jumlah: 15
  }
]

```

Penjelasan:

Sintaks ini berfungsi memfilter data pada bulan Juni 2026, menggabungkannya dengan target kualitas, lalu menghitung persentase cacat dan status (OK/NOT OK). Dengan \$facet, query menghasilkan tiga ringkasan sekaligus: total batch, rata-rata cacat, dan 3 mesin dengan kasus NOT OK terbanyak.

Latihan Mandiri

1. Database baru, buat koleksi data_sensor dengan 5000 dokumen (field sensor_id, nilai, timestamp, category).
2. Pipeline untuk:

- Menampilkan rata-rata nilai per sensor_id pada 7 hari terakhir.
 - Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan \$group dengan \$push atau \$sum kondisional).
 - Menambahkan field jam dan menghitung nilai rata-rata per jam per sensor.
3. Buat koleksi sensor_info lalu join untuk menampilkan nama sensor dalam hasil.

1. Pipeline Tugas Mandiri:

```
use db_sensor_baru

db.createCollection("data_sensor")

// generate 5000 dokumen
let categories = ["A", "B", "C"]

let bulk = []

for (let i = 0; i < 5000; i++) {
  bulk.push({
    sensor_id: "S" + (Math.floor(Math.random() * 10) + 1), // S1 - S10
    nilai: Math.floor(Math.random() * 101), // 0 - 100
    timestamp: new Date(Date.now() - Math.random() * 7 * 24 * 60 * 60 * 1000), // 7
    hari terakhir
    category: categories[Math.floor(Math.random() * categories.length)]
  })
}

db.data_sensor.insertMany(bulk)
```

Output:

```
'4968': ObjectId('69f8637f9d8314a114d62930'),
'4969': ObjectId('69f8637f9d8314a114d62931'),
'4970': ObjectId('69f8637f9d8314a114d62932'),
'4971': ObjectId('69f8637f9d8314a114d62933'),
'4972': ObjectId('69f8637f9d8314a114d62934'),
'4973': ObjectId('69f8637f9d8314a114d62935'),
```

Penjelasan:

Bertujuan untuk menambahkan 5000 data dummy ke dalam database untuk percobaan kedepan.

2. Rata-Rata Nilai Sensor pada 7 Hari Terakhir

```
db.data_sensor.aggregate([
  {
    $match: {
      timestamp: {
        $gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000)
      }
    }
  }
])
```

```

    },
    {
      $group: {
        _id: "$sensor_id",
        avg_nilai: { $avg: "$nilai" }
      }
    }
  ]
})

```

Output:

```

_id: 'S7',
avg_nilai: 49.66595744680851
}
{
_id: 'S4',
avg_nilai: 50.268389662027836
}
{
_id: 'S3',
avg_nilai: 51.111111111111114
}
{
_id: 'S5',
avg_nilai: 48.942184154175585
}
}

```

Penjelasan:

Pipeline ini memfilter data 7 hari terakhir dengan \$match, lalu mengelompokkan berdasarkan sensor_id untuk menghitung rata-rata nilai tiap sensor.

3. Tambah Field Jam & Rata-Rata per Jam per Sensor

```

db.data_sensor.aggregate([
  {
    $addFields: {
      jam: { $hour: "$timestamp" }
    }
  },
  {
    $group: {
      _id: {
        sensor_id: "$sensor_id",
        jam: "$jam"
      },
      avg_per_jam: { $avg: "$nilai" }
    }
  },
  { $sort: {
    "_id.sensor_id": 1,
    "_id.jam": 1
  }
}
]
)

```

```
}
} ])
```

Output:

```
_id: {
  sensor_id: 'S1',
  jam: 0
},
avg_per_jam: 44.45454545454545
}
{
  _id: {
    sensor_id: 'S1',
    jam: 1
  },
  avg_per_jam: 43.0625
}
{
  _id: {
    sensor_id: 'S1',
    jam: 2
  },
  avg_per_jam: 39.91304347826087
}
{
  _id: {
    sensor_id: 'S1',
    jam: 3
  },
  avg_per_jam: 50.15384615384615
}
{
  _id: {
    sensor_id: 'S1',
    jam: 4
  },
  avg_per_jam: 60.44
}
{
  _id: {
    sensor_id: 'S1',
    jam: 5
  },
  avg_per_jam: 50.15
}
{
  _id: {
    sensor_id: 'S1',
    jam: 6
  },
  avg_per_jam: 47.23076923076923
}
{
  _id: {
    sensor_id: 'S1',
    jam: 7
  },
  avg_per_jam: 56.875
}
{
  _id: {
    sensor_id: 'S1',
    jam: 8
  },
  avg_per_jam: 48.6
}
{
  _id: {
    sensor_id: 'S1',
    jam: 9
  },
  avg_per_jam: 46.473684210526315
}
{
  _id: {
    sensor_id: 'S1',
    jam: 10
  },
  avg_per_jam: 55.55555555555556
}
{
  _id: {
    sensor_id: 'S1',
    jam: 11
  },
  avg_per_jam: 47.78260869565217
}
{
  _id: {
    sensor_id: 'S1',
    jam: 12
  },
  avg_per_jam: 50.22222222222222
}
{
  _id: {
    sensor_id: 'S1',
    jam: 13
  },
  avg_per_jam: 55.77777777777778
}
```

Penjelasan:

Pipeline ini menambahkan field jam dari timestamp, lalu mengelompokkan data berdasarkan sensor_id dan jam untuk menghitung rata-rata nilai per jam, kemudian hasilnya diurutkan agar rapi.

4. Koleksi Sensor

a. Buat Koleksi

```
db.createCollection("sensor_info")

db.sensor_info.insertMany([
  { sensor_id: "S1", nama_sensor: "Sensor Suhu 1" },
  { sensor_id: "S2", nama_sensor: "Sensor Tekanan 1" },
  { sensor_id: "S3", nama_sensor: "Sensor Kelembaban 1" },
  { sensor_id: "S4", nama_sensor: "Sensor Suhu 2" },
  { sensor_id: "S5", nama_sensor: "Sensor Tekanan 2" },
  { sensor_id: "S6", nama_sensor: "Sensor Kelembaban 2" },
  { sensor_id: "S7", nama_sensor: "Sensor Getaran 1" },
  { sensor_id: "S8", nama_sensor: "Sensor Getaran 2" },
  { sensor_id: "S9", nama_sensor: "Sensor Cahaya 1" },
```

```
{ sensor_id: "S10", nama_sensor: "Sensor Cahaya 2" }
])
```

b. Join \$lookup untuk Menampilkan Sensor

```
db.data_sensor.aggregate([
  {
    $group: {
      _id: "$sensor_id",
      avg_nilai: { $avg: "$nilai" }
    }
  },
  {
    $lookup: {
      from: "sensor_info",
      localField: "_id",
      foreignField: "sensor_id",
      as: "info"
    }
  },
  { $unwind: "$info" },
  {
    $project: {
      sensor_id: "$_id",
      nama_sensor: "$info.nama_sensor",
      avg_nilai: 1,
      _id: 0
    }
  }
])
```

Output:

```
{
  avg_nilai: 52.185110663983906,
  sensor_id: 'S9',
  nama_sensor: 'Sensor Cahaya 1'
}
{
  avg_nilai: 50.431640625,
  sensor_id: 'S2',
  nama_sensor: 'Sensor Tekanan 1'
}
{
  avg_nilai: 51.724,
  sensor_id: 'S8',
  nama_sensor: 'Sensor Getaran 2'
}
{
  avg_nilai: 48.942184154175585,
  sensor_id: 'S5',
  nama_sensor: 'Sensor Tekanan 2'
}
{
  avg_nilai: 51.111111111111114,
  sensor_id: 'S3',
  nama_sensor: 'Sensor Kelembaban'
}
```

Penjelasan:

Penggunaan stage \$lookup pada pipeline aggregation berfungsi untuk melakukan join antar koleksi di MongoDB, dalam hal ini menggabungkan data dari koleksi data_sensor dengan sensor_info berdasarkan field yang sama yaitu sensor_id. Melalui mekanisme ini, setiap dokumen hasil agregasi yang sebelumnya hanya berisi identitas sensor dan nilai perhitungannya dapat diperkaya dengan informasi tambahan seperti nama_sensor.

KESIMPULAN

Pembahasan praktikum ini mencakup pemahaman dan penerapan Aggregation Framework MongoDB untuk analisis data produksi dan kualitas, penggunaan pipeline untuk menyederhanakan proses pengolahan data dari mentah menjadi terstruktur, serta pemanfaatan berbagai tahapan agregasi untuk penyaringan, pengelompokan, perhitungan metrik, dan penggabungan data. Praktikum ini juga membahas analisis data sensor berdasarkan waktu dan identitas sensor untuk mengetahui rata-rata nilai dalam periode tertentu, mengidentifikasi kemunculan nilai tinggi secara berulang sebagai indikasi anomali, serta menganalisis pola distribusi data secara lebih rinci per waktu. Selain itu, integrasi antar koleksi digunakan untuk memperkaya hasil analisis dengan informasi tambahan sehingga lebih kontekstual. Secara keseluruhan, pendekatan ini menunjukkan bahwa pengolahan dan analisis data dapat dilakukan secara efisien, terstruktur, dan langsung di dalam database untuk mendukung pengambilan keputusan yang lebih akurat.